One-Dimensional Packing of Dance Equipment

Abstract

In the realm of university dance competitions across the US, Raas has evolved into a prominent multicultural intercollegiate dance community. Among other competitors, UW Raas is a student led organization representing the University of Washington at Raas competitions nationwide. Each team designs and travels with elaborate sets to fit their performance themes, facing significant logistical challenges in transporting and assembling these sets within stringent airline and venue constraints. This paper presents a mathematical model to create a blueprint for the set design and a packing configuration that fulfills UW Raas' requirements given weight, storage, and dimension constraints. The model ensures that the required design is achieved using available pipes from the team's inventory and efficiently packs them into skibags, adhering to weight, dimension, and cost restrictions. The integer and linear programming model uses binary decision variables for the possible locations in each bag for different pipe lengths, ensuring that each bag is packed in a way that minimizes the number of bags required and optimizes the use of each bag's volume. A solution was found using these methods which reduces travel costs and ensures balanced packing, providing a practical solution to the logistical challenges faced by Raas teams.

1 Introduction

In the realm of university dance competitions across the USA, Raas has grown into one of the biggest multicultural intercollegiate dance communities in the country. Amongst the competitors, UW Raas, representing the University of Washington, travels all over the nation participating in Raas competitions.

Raas competitions are hosted by universities on their campuses and participating teams fly in over the weekend to compete. Teams choose a theme for their performance each year, which they incorporate into their routine as well as the set design. The set is one of the biggest artistic components of Raas performances. Each team designs their set to fit the theme, flies in with the necessary parts of the set and builds it during the competition weekend. However, each team faces a significant logistical challenge of transporting and assembling such elaborate sets.

The size of the set design is restricted by the storage space allotted to each team at the venue. The volume, weight, and cost of components of the set is

restricted by the size of bags they are packed into, the cost of checking in those bags, and the weight limit of checked-in bags imposed by airlines.

This year, UW Raas' theme is Subway Surfers. Their set consists of train outlines made from PVC pipes. The specific outlines of the trains and their dimensions will be referred to as "designs." For each design, UW Raas needs instructions on how to assemble each design (these instructions will be referred to as "blueprints"). They have an existing inventory of PVC pipes as well as skibags (cylindrical soft bags of fixed cross-sectional area) to carry these pipes to competitions in. The goal is as follows: given a design of trains and an inventory of PVC pipes, to provide a blueprint for building the trains along with a configuration to pack the skibags with PVC pipes that fulfill the team's needs while meeting weight, storage, and dimension constraints.

2 Data Collection

2.1 Train Design

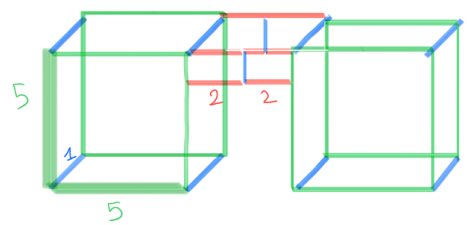

Figure 1: Train 1

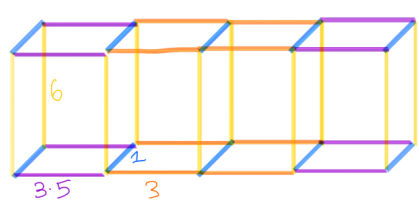

Figure 2: Train 2

For each train's design in figures 1 and 2, sides are color coded according to their lengths. Sides are defined as the uninterrupted segments of pipes in the designs (in figures 1 and 2, each individually colored segment is a side). Each train has a 3 dimensional design where the face of the train is mirrored at the back, with the front and back connected by one foot pipes at various locations as shown in the diagrams. Hence, for example, in figure 1, it can be seen that

Side Length(ft)	Number Needed in Designs
1	20
2	8
3	8
3.5	8
4	0
5	16
6	10

Table 1: The side lengths and quantities of each side length in the designs.

there are ten sides of length 1'. Table 1 describes the total number of sides of each length present in both designs.

2.2 skibag Dimensions

Figure 3 shows a dimensioned drawing of the skibags used by the Raas team, with essential dimensions and the number of bags owned by the team listed after. Note that each skibag may accommodate 21 pipes of length 6' laying side by side, lengthwise in the bags.

Figure 3: skibag

- skibag dimensions: 9.49 x7.17 x 2.56 inches
- skibag quantity: 4

2.3 Inventory of Pipes

Table 2 details the number of pipes of each length available to the Raas dance team. Note that all pipes have a diameter of 1" and a density of 0.35 lb/ft.

Pipe Length (ft)	Inventory
1	35
2	13
3	18
3.5	8
4	10
5	16
6	4

Table 2: The team's inventory of pipes of various lengths.

3 Set Up for Design Blueprint

The mathematical model which was designed to solve this problem first handles the design blueprint, and takes as inputs the number of pipes available in the team's inventory and their lengths as well as the number of sides of each length required in the designs. From this information, the model creates constraints which ensure that the pipes included in the solution actually combine to form the needed design.

In order to model this, the decision variables $p_{l,n,s}$ were used. The value of a specific $p_{l,n,s}$ is 1 if there are n pipes of length l in the blueprint for s, a given side of the design, and is 0 otherwise. With this representation, it is possible to parameterize the problem to create desired objective functions and constraints which match the physical requirements of the system.

In this problem, a couple variables that will be used are defined as follows.

- Let S be the total number of sides (uninterrupted line segments) that are in each design (70 in this context).
- Let s index each side of both designs (s takes values 1 through S).
- Let $len(s)$ be the length of side s in the design.
- Let L be the number of lengths of pipes that are available (6 in this context).
- Let l index each pipe length (l takes values 1 through L).
- Let $a(l)$ be the number of available pipes of length l (listed in table 2).
- Let $len(l)$ be the pipe length with index l (lengths are indexed from lowest being 1 to highest)
- Let Ns be the maximum number of pipes of the shortest length that could fit into a single side (in this context, 6).
- Let n be a number of pipes (n takes values 1 through Ns).

3.1 Constraints for Design Blueprint

1. First, an implicit constraint will be explained. Since the binary variable $p_{l,n,s}$ represents the presence of n pipes of length l in a specific side, it would not make sense to have, for instance, b_{111} and b_{121} both be equal to 1, as this would mean that there is 1 pipe of length 1 present in side 1 and that there are also 2 pipes of length 1 present in side 1. As such, the following constraint will be used to address this.

$$\sum_{n=1}^{N_s} p_{l,n,s} \leq 1 \quad \text{for each} \quad 1 \leq l \leq L, 1 \leq s \leq S$$

   This guarantees that the number of pipes of a specific length in a specific side of the design is only defined once, and hence this constraint avoids contradicting quantities of pipes.

2. Next, it is important to quantify that the length of each of the sides of the design is exactly met by the pipes that contribute to it in the solution. The following constraint is included to implement this need.

$$\sum_{l=1}^{L} \sum_{n=1}^{N_s} p_{l,n,s} \cdot \text{len}(l) \cdot n = \text{len}(s) \quad \text{for each} \quad 1 \leq s \leq S$$

   This constraint ensures that the blueprint generated exactly matches the lengths of the desired design.

3. This constraint is implemented to guarantee that the number of available pipes in the team's inventory is not exceeded by the blueprint in attempting to meet the design.

$$\sum_{s=1}^{S} \sum_{n=1}^{N_s} p_{l,n,s} \cdot n \leq \text{a}(l) \quad \text{for each} \quad 1 \leq l \leq L$$

4 Set Up for Pipe Packing

For the pipe packing, some context is necessary on how the space available in each bag was discretized. Since each skibag is a cylindrical bag with a constant cross section when packed and all of the pipes have the same diameter, the skibags were discretized into rows of pipes. Specifically, it was found that each skibag can contain 21 rows of pipes laying lengthwise in the bag (this was verified, as a skibag was found to be able to hold 21 pipes or rows of pipes 6' in length). Hence, each bag was discretized into 21 rows where pipes could be located, each having a length of 6'. This greatly simplifies the packing considerations, as it transforms the three-dimensional packing problem into a one-dimensional packing problem.

To translate the packing considerations into a mathematical model that can then be optimized, we relied on binary variables to represent the presence of pipes in different states. Specifically, the variable $b_{l,n,i,j}$ is equal to 1 if n pipes of length l are present in the ith row of skibag j. Additionally, two binary indicator variables were included in the program. The variable $bInUse_j$ is equal to 1 if bag j is in use (has pipes in it) and is 0 otherwise. The variable $bRowInUse_{i,j}$ is equal to 1 if row i of bag j is in use and is 0 otherwise. With this representation, it is possible to parameterize the problem to create desired objective functions and constraints which match the physical requirements of the system.

In this problem we will define a couple variables to use in our equations as follows in addition to the variables defined in section 3 (note that n is redefined in this context).

- Let M be the total number of rows in a bag (21 in this context).
- Let i index each row of each bag (i takes values 1 through M).
- Let J be the total number of bags available (4 in this context).
- Let j index each bag (j takes values 1 through J).
- Let N be the maximum number of pipes of the shortest length that could fit into a single row in a skibag (6 in this context).
- Let n be a number of pipes (n takes values 1 through N).
- Let L_{MAX} be the maximum length of a row (6' in this context).
- Let ρ be the length density of the pipes in lb/ft (0.35 in this context).
- Let $W_{bag,empty}$ be the empty weight of a skibag in lbs (2 in this context).

4.1 Constraints for Pipe Packing

1. First, an implicit constraint will be explained. Since the binary variable $b_{l,n,i,j}$ represents the presence of n pipes of length l in a specific row, it would not make sense to have, for instance, $b_{1,1,1,1}$ and $b_{1,2,1,1}$ both be equal to 1, as this would mean that there is 1 pipe of length 1 present in row 1 of bag 1 and that there are also 2 pipes of length 1 present in row 1 of bag 1. As such, the following constraint will be used to address this.

$$\sum_{n=1}^{N} b_{l,n,i,j} \leq 1 \quad \text{for each} \quad 1 \leq i \leq M, 1 \leq j \leq J, 1 \leq l \leq L$$

   This constraint guarantees that the number of pipes of a specific length present in a specific row of a specific bag is only defined once, ensuring that no contradictions are allowed in the solution.

2. The next set of constraints identifies the relationship between the indicator variables and the decision variables. This relationship is easily understood. If the indicator variable $bInUse_j$ is equal to zero, meaning that bag j is not being used, then the decision variable $b_{l,n,i,j}$ must also be equal to zero. This logic applies similarly to the indicator variable representing whether the rows in each bag are used or not. These constraints are represented mathematically as below.

$$bInUse_j = 0 \iff \sum_{i=1}^{I}\sum_{n=1}^{N}\sum_{l=1}^{L} b_{l,n,i,j} = 0 \quad \text{for each} \quad 1 \leq j \leq J$$

$$bRowInUse_{i,j} = 0 \iff \sum_{n=1}^{N}\sum_{l=1}^{L} b_{l,n,i,j} = 0 \quad \text{for each} \quad 1 \leq i \leq I, 1 \leq j \leq J$$

These constraints guarantee that the intended relationship between the indicator variables and the decision variables is maintained.

3. Next, the spatial constraints presented in the packing problem will be addressed. In each skibag, there is room for M rows of pipes that cannot exceed a length of L_{MAX}. In order to represent this constraint, the variable i will take integer values from 1 to M, allowing for each of the M rows to be accounted for. Then, for each value of i and j (for each row i in each bag j), the following constraint will be placed.

$$\sum_{n=1}^{N}\sum_{l=1}^{L} b_{l,n,i,j} \cdot n \cdot \text{len}(l) \leq L_{MAX} \quad \text{for each} \quad 1 \leq i \leq M, 1 \leq j \leq J$$

The left side of this constraint represents the total length of each row of pipes in each skibag and dictates that these lengths may not exceed the length of the skibags.

4. A weight constraint must also be placed on the packing configurations, as each bag may not exceed the weight allowed by the airline of choice of the dance team. To guarantee this, the weights of the pipes in each bag may not (in addition to the weight of the bag) exceed 45 lbs. The weights of the pipes (as all pipes have the same cross-sectional and material properties) may be found by scaling the pipe lengths by a density, ρ [lb/ft]. Hence, the following constraint is implemented to ensure that no overweight bags are present in the solution.

$$\sum_{i=1}^{M}\sum_{n=1}^{N}\sum_{l=1}^{L} b_{l,n,i,j} \cdot n \cdot \text{len}(l) \cdot \rho \leq 45 - W_{bag,empty} \quad \text{for each} \quad 1 \leq j \leq J$$

The left-hand side represents the weight of the pipes in bag j in pounds, and the inequality dictates that this weight may not exceed 45 pounds minus the weight of the empty bag (also in pounds).

5 Combining the Design and Packing Considerations

Logically, it is obvious that the same pipes that are being used in the set designs need to be packed, however, since two different variables are used ($p_{l,n,s}$ for the pipes in the set design and $b_{l,n,i,j}$ for the bag packing), the mathematical model needs explicit constraints to handle this relationship between the variables.

5.1 Constraints for Combining the Design and Packing Considerations

1. To fulfill the requirements of meshing the design and packing considerations, constraints must be included as follows to guarantee that the same number of pipes of each length are present in both the set design and the packing.

$$\sum_{j=1}^{J}\sum_{i=1}^{M}\sum_{n=1}^{N} b_{l,n,i,j} \cdot n = \sum_{n=1}^{Ns}\sum_{s=1}^{S} p_{l,n,s} \cdot n \quad \text{for each} \quad 1 \leq l \leq L$$

   This constraint guarantees that the two portions of the math model work together, solving the real problem of packing the same pipes that are used in the dance set design.

6 Analysis

With the constraints considered and in place, it is now necessary to define an objective function which will accomplish our goals analytically. For the sake of this problem, there are three ideals to be considered.

1. The number of bags required for the dance team to take should be minimized to make the cost of travel minimal.

2. The number of rows used in each bag should be minimized (a bag should be packed such that each layer uses as much of the cross-sectional area as possible to balance the bag properly).

3. The number of pipes (n_{pipes}) should also be minimized (with less urgency than the other considerations) in order to save the dance team time and energy in assembling their set design.

Each of these objective functions necessitates manipulating the binary decision variable to keep track of other quantities. The first consideration of the objective function will be discussed first to demonstrate this and specify a solution.

To define the number of bags used in a solution (n_{Bags}), the indicator variables $bInUse_j$ can be summed.

$$n_{Bags} = \sum_j bInUse_j$$

Using these methods, the number of bags used is tracked and can thus be weighted into the objective function in an attempt to minimize the bag usage.

The second consideration in the objective function can then be implemented similarly with a counter function which identifies the number of rows used (n_{Rows}) which is to be minimized.

$$n_{Rows} = \sum_j \sum_i bRowInUse_{i,j}$$

The third consideration in the objective function can be implemented by counting the number of pipes either using the $p_{l,n,s}$ variables or the $b_{l,n,i,j}$ variables. The method described below which was used in the model counts the pipes used by using the $p_{l,n,s}$ variables.

$$n_{Pipes} = \sum_s \sum_l \sum_n p_{l,n,s} \cdot n$$

With these quantities defined, the objective can be stated simply as minimizing the objective function (O) below. For the provided solution, weights were chosen such that the number of bags would be minimized with ultimate priority and the number of rows would be minimized with slightly higher priority than the number of pipes.

$$O = \alpha_1 n_{Bags} + \alpha_2 n_{Rows} + \alpha_3 n_{Pipes} \quad \alpha_1 = 1000, \alpha_2 = 10, \alpha_3 = 1$$

7 Solution

Using the optimization function, the following solution was found. Table 3 describes the design blueprint portion of the solution. First, the table lists the number of sides needed of each total length in the designs. Then, the table lists different specific pipe buildups for each total length as well as how many sides of each specific buildup were output by the solution. This provides all of the necessary information for the design blueprint.

Side Length	Number Needed	Pipes in Buildup	Number of Sides of Buildup
1'	20	1'	20
2'	8	2'	8
3'	8	3'	8
3.5'	8	3.5'	8
5'	16	5'	16
6'	10	6'	4
		2' & 4'	3
		3' & 3'	3

Table 3: The design blueprint results from the model.

Table 4 describes the pipe packing portion of the solution. First, the table identifies the bag into which the pipes will be packed. Then, the table lists the number of rows containing each specific combination of pipes in each bag. Next, the table lists the specific combinations of pipes that are to be packed into each row. This table provides all of the necessary information for the pipe packing. Finally, the table lists the total weight of each packed bag as well as the percent of the available volume of each bag that was used.

Bag	Number of Rows	Pipes	Total Weight [lbs]	Volume Used
1	13	1' & 5'	39.5	85%
	2	6'		
	1	1' & 1' & 3.5'		
	1	2' & 3.5'		
	1	2' & 4'		
2	6	2' & 3.5'	44.4	96%
	6	3' & 3'		
	3	1' & 5'		
	2	1' & 2' & 3'		
	2	6'		
	1	2' & 4'		
	1	4'		

Table 4: The pipe packing results from the model.

Compared to previous solutions from the Raas dance team which involved taking 3 or 4 bags to each competition, this new solution is much more optimized. Requiring only 2 bags for each competition, the solution found using the methods described heavily benefits the team. Not only will this solution save the team a little money in travel costs, but the solution will also make the pipes easier to transport with fewer bags to keep track of.

8 Further Extensions

One of the important parts that we had in consideration was how we could extend this model to help with other packing problems that UW Raas might encounter. In order to account for this, we made sure to write our model in terms of variables that could be adjusted. For example, the values of M, J, L, and N can all be changed in order to fit new types of packing problems. If the team decides that they would like to pack something else, they can easily resume our program for those purposes.

9 Application to Other Domains

This packing problem is not just limited to UW Raas'. This model could be very helpful in a number of other fields beyond dance competition. For example,

9.1 Construction and Manufacturing

There are often pipes, beams, or other materials that are of cylindrical shape that need to be packed. It is often ideal to want to pack them as closely together as possible without violating a weight limit (restricted by maybe the truck load or how much a person could carry). Optimizing this packing could lead to reduced labor and faster construction times.

9.2 Grocery Store/Warehouse

Another place that this could be used is in warehouses or grocery stores. Often they have to pack items such that they are stored effectively and are possible to move with some weight constraint. This could be helpful in minimizing the space needed for storing a selection of items.

9.3 Other RSOs/Events

I believe that many other RSOs that set up things like tents, posters, and other small scale visuals could make great use of this math model. They are often trying to build this object that is supposed to take up a lot of surface area, and trying to find a way to pack its components so that it can be set up and torn down quickly. This is very similar to the UW Raas' needs of this math model. Other RSOs could just plug in the dimensions of their components and get out a packing of it. It would take some more interpretative work to figure out the exact blueprint.

10 Conclusion

In conclusion, our mathematical modeling successfully addressed the logistical challenges faced by UW Raas in transporting and assembling their elaborate

set designs for dance competitions. By formulating an Integer and Linear Programming model for both the design blueprint and pipe packing, we were able to optimize the utilization of available resources while meeting weight, storage, and dimension constraints. Our model minimizes the number of bags required for travel and optimizes the packing configuration within each bag. These models not only streamline the logistical process for UW Raas but can also be further applied to space optimization problems in construction and manufacturing firms, grocery stores or even for other dance teams.

11 Acknowledgement

We would like to extend our heartfelt gratitude to UW Raas for partnering with us on this project and providing valuable time and insights. A special thank you to Neeti Agarwal, the creative captain of UW Raas, for meeting with us and sharing her designs. We are also deeply thankful to Professor Sarah Billey for her unwavering support, guidance, and teaching throughout this endeavor. Additionally, we appreciate the feedback from Charles Chen, Everett Hirano, Logan Hosoda, Cade Tanaka, Haoran Xiang, which significantly improved our draft.

12 References

Packing problems, especially pipe packing problems, are a common problem that has been modeled in various different contexts in the past. Our problem can be most closely linked to construction and manufacturing firms that deal with similar pipe storage problems every day.

The first paper we referenced was Orbak, *et al.* (2023) [2]. This problem "analyzed the optimized stacking sequence and layout of containers" storing pipes. The model used a binary variable to track if there is any pipe of type i placed in a container of type j, assigning it a value of 1 if the pipe is present and 0 otherwise. We drew inspiration from this model to track the storage of PVC pipes of specific lengths in skibags, allowing us to assign binary variables that tracked which pipes go into which skibags. This also makes it easier to draw inference from the model when providing the team with a packing design as well as makes the model easily replaceable for future packing problems for the team.

The second source we referenced was Doerr and Levasseur (n.d.), *Partitions of Sets and the Law of Addition* [1]. UW Raas has an existing inventory of pipes, from which a subset of pipes will be used for the set, which can further be classified into smaller subsets to represent each bag. One subset could be of pipes of the same lengths, or another could be pipes required to build one train in the set. The way we choose to create subsets of the inventory influences the organization of skibags. Hence, we referenced open-source material to learn more about set partitions and their applicability in packing problems.

13 Bibliography

1. Doerr, Al, and Ken Levasseur. "Partitions of Sets and the Law of Addition."

2. Orbak, Â.Y.; Küçük, M.; Akansel, M.; Sharma, S.; Li, C.; Kumar, R.; Singh, S.; Di Bona, G. Mathematical Model Assisted Six-Sigma Approach for Reducing the Logistics Costs of a Pipe Manufacturing Company: A Novel Experimental Approach. Mathematics 2023, 11, 621. https://doi.org/10.3390/math11030621

14 Appendix

The following is the output directly from the code which was interpreted in the solutions section for ease of reading. This is then followed by the code which was used to implement the model and find the solution.

```
Set Design Blueprint:

20 sides of length 1' are needed
in the design. These sides are made
up of the following combinations of
pipes:
    ------------------------------
    1 pipe(s) of length 1'
    ------------------------------
    1 pipe(s) of length 1'
    ------------------------------
    1 pipe(s) of length 1'
    ------------------------------
    1 pipe(s) of length 1'
    ------------------------------
    1 pipe(s) of length 1'
    ------------------------------
    1 pipe(s) of length 1'
    ------------------------------
    1 pipe(s) of length 1'
    ------------------------------
    1 pipe(s) of length 1'
    ------------------------------
    1 pipe(s) of length 1'
    ------------------------------
    1 pipe(s) of length 1'
    ------------------------------
    1 pipe(s) of length 1'
    ------------------------------
    1 pipe(s) of length 1'
    ------------------------------
    1 pipe(s) of length 1'
    ------------------------------
    1 pipe(s) of length 1'
    ------------------------------
    1 pipe(s) of length 1'
    ------------------------------
    1 pipe(s) of length 1'
    ------------------------------
    1 pipe(s) of length 1'
    ------------------------------
    1 pipe(s) of length 1'
    ------------------------------
    1 pipe(s) of length 1'
    ------------------------------
    1 pipe(s) of length 1'
    ------------------------------
    1 pipe(s) of length 1'
    ------------------------------
    1 pipe(s) of length 1'
    ------------------------------
    1 pipe(s) of length 1'
    ------------------------------
8 sides of length 2' are needed
in the design. These sides are made
up of the following combinations of
pipes:
    ------------------------------
    1 pipe(s) of length 2'
    ------------------------------
    1 pipe(s) of length 2'
```

```
    ------------------------------
    1 pipe(s) of length 2'
    ------------------------------
    1 pipe(s) of length 2'
    ------------------------------
    1 pipe(s) of length 2'
    ------------------------------
    1 pipe(s) of length 2'
    ------------------------------
    1 pipe(s) of length 2'
    ------------------------------
    1 pipe(s) of length 2'
    ------------------------------
8 sides of length 3' are needed
in the design. These sides are made
up of the following combinations of
pipes:
    ------------------------------
    1 pipe(s) of length 3'
    ------------------------------
    1 pipe(s) of length 3'
    ------------------------------
    1 pipe(s) of length 3'
    ------------------------------
    1 pipe(s) of length 3'
    ------------------------------
    1 pipe(s) of length 3'
    ------------------------------
    1 pipe(s) of length 3'
    ------------------------------
    1 pipe(s) of length 3'
    ------------------------------
    1 pipe(s) of length 3'
    ------------------------------
8 sides of length 3.5' are needed
in the design. These sides are made
up of the following combinations of
pipes:
    ------------------------------
    1 pipe(s) of length 3.5'
    ------------------------------
    1 pipe(s) of length 3.5'
    ------------------------------
    1 pipe(s) of length 3.5'
    ------------------------------
    1 pipe(s) of length 3.5'
    ------------------------------
    1 pipe(s) of length 3.5'
    ------------------------------
    1 pipe(s) of length 3.5'
    ------------------------------
    1 pipe(s) of length 3.5'
    ------------------------------
16 sides of length 5' are needed
in the design. These sides are made
up of the following combinations of
pipes:
    ------------------------------
    1 pipe(s) of length 5'
    ------------------------------
    1 pipe(s) of length 5'
    ------------------------------
    1 pipe(s) of length 5'
    ------------------------------
    1 pipe(s) of length 5'
    ------------------------------
    1 pipe(s) of length 5'
    ------------------------------
    1 pipe(s) of length 5'
    ------------------------------
    1 pipe(s) of length 5'
    ------------------------------
    1 pipe(s) of length 5'
    ------------------------------
    1 pipe(s) of length 5'
    ------------------------------
    1 pipe(s) of length 5'
    ------------------------------
    1 pipe(s) of length 5'
    ------------------------------
    1 pipe(s) of length 5'
    ------------------------------
    1 pipe(s) of length 5'
    ------------------------------
    1 pipe(s) of length 5'
    ------------------------------
    1 pipe(s) of length 5'
    ------------------------------
    1 pipe(s) of length 5'
    ------------------------------
```

```
10 sides of length 6' are needed
in the design. These sides are made
up of the following combinations of
pipes:
    ----------------------------
    2 pipe(s) of length 3'
    ----------------------------
    1 pipe(s) of length 6'
    ----------------------------
    1 pipe(s) of length 6'
    ----------------------------
    1 pipe(s) of length 6'
    ----------------------------
    1 pipe(s) of length 2'
    1 pipe(s) of length 4'
    ----------------------------
    1 pipe(s) of length 2'
    1 pipe(s) of length 4'
    ----------------------------
    2 pipe(s) of length 3'
    ----------------------------
    1 pipe(s) of length 6'
    ----------------------------
    2 pipe(s) of length 3'
    ----------------------------
    1 pipe(s) of length 2'
    1 pipe(s) of length 4'
    ----------------------------

Bag Packing List and Configuration:

-----------------------------
20.0 pipes of length 1'
11.0 pipes of length 2'
14.0 pipes of length 3'
8.0 pipes of length 3.5'
3.0 pipes of length 4'
16.0 pipes of length 5'
4.0 pipes of length 6'
-----------------------------
-----------------------------
Bag 1:

Total Weight: 39.5 pounds
Percent of Volume Used: 85%
```

```
Row 1
    1 pipe(s) of length 2'
    1 pipe(s) of length 3.5'
Row 2
    1 pipe(s) of length 1'
    1 pipe(s) of length 5'
Row 3
    1 pipe(s) of length 1'
    1 pipe(s) of length 5'
Row 4
    1 pipe(s) of length 1'
    1 pipe(s) of length 5'
Row 5
    2 pipe(s) of length 1'
    1 pipe(s) of length 3.5'
Row 6
    1 pipe(s) of length 1'
    1 pipe(s) of length 5'
Row 7
    1 pipe(s) of length 1'
    1 pipe(s) of length 5'
Row 8
    1 pipe(s) of length 6'
Row 9
    1 pipe(s) of length 1'
    1 pipe(s) of length 5'
Row 10
    1 pipe(s) of length 2'
    1 pipe(s) of length 4'
Row 11
    1 pipe(s) of length 1'
    1 pipe(s) of length 5'
Row 12
    1 pipe(s) of length 6'
Row 13
    1 pipe(s) of length 1'
    1 pipe(s) of length 5'
Row 14
    1 pipe(s) of length 1'
    1 pipe(s) of length 5'
Row 15
    1 pipe(s) of length 1'
    1 pipe(s) of length 5'
Row 16
    1 pipe(s) of length 1'
    1 pipe(s) of length 5'
```

```
    Row 17
        1 pipe(s) of length 1'
        1 pipe(s) of length 5'
    Row 18
        1 pipe(s) of length 1'
        1 pipe(s) of length 5'
-----------------------------
Bag 2:

Total Weight: 44.4 pounds
Percent of Volume Used: 96%

    Row 1
        1 pipe(s) of length 1'
        1 pipe(s) of length 5'
    Row 2
        1 pipe(s) of length 2'
        1 pipe(s) of length 3.5'
    Row 3
        1 pipe(s) of length 2'
        1 pipe(s) of length 3.5'
    Row 4
        1 pipe(s) of length 1'
        1 pipe(s) of length 2'
        1 pipe(s) of length 3'
    Row 5
        1 pipe(s) of length 2'
        1 pipe(s) of length 4'
    Row 6
        2 pipe(s) of length 3'
    Row 7
        1 pipe(s) of length 4'
    Row 8
        1 pipe(s) of length 2'
        1 pipe(s) of length 3.5'
    Row 9
        1 pipe(s) of length 2'
        1 pipe(s) of length 3.5'
    Row 10
        2 pipe(s) of length 3'
    Row 11
        1 pipe(s) of length 1'
        1 pipe(s) of length 2'
        1 pipe(s) of length 3'
    Row 12
        2 pipe(s) of length 3'
    Row 13
        1 pipe(s) of length 2'
        1 pipe(s) of length 3.5'
    Row 14
        1 pipe(s) of length 2'
        1 pipe(s) of length 3.5'
    Row 15
        2 pipe(s) of length 3'
    Row 16
        1 pipe(s) of length 6'
    Row 17
        2 pipe(s) of length 3'
    Row 18
        1 pipe(s) of length 6'
    Row 19
        1 pipe(s) of length 1'
        1 pipe(s) of length 5'
    Row 20
        1 pipe(s) of length 1'
        1 pipe(s) of length 5'
    Row 21
        2 pipe(s) of length 3'
-----------------------------
```

```
import math
import numpy as np
import gurobipy as gp
from gurobipy import GRB
import matplotlib.pyplot as pyplot
import csv
```

```
#determines number of pipes to bring given input design
z=gp.Model("Z")


#defining inputs
#_____________________________________


#for first program
s1  = 20             # number of sections of design of length 1
s2  = 8              # number of sections of design of length 2
s3  = 8              # number of sections of design of length 3
s35 = 8              # number of sections of design of length 3.5
s4  = 0              # number of sections of design of length 4
s5  = 16             # number of sections of design of length 5
s6  = 10             # number of sections of design of length 6
Lens=[1,2,3,3.5,4,5,6]        #list of available pipe lengths in ascending order
avail=[35,13,18,8,10,16,4]    #list of lengths of pipes in same order as "Lens"


#for second program
N = 6          # maximum number of pipes that could fit in a bag
I = 21         # maximum number of rows of pipes in a bag
J = 4          # maximum number of bags allowed
Lmax = 6       # length of skibags in feet
Wmax = 45      # max weight of bag defined by airliner in lbs
Wempty = 2     # empty bag weight in lbs
rho = 0.35     # density of pipes in lb/ft


#_____________________________________
#determine vars from inputs

sideNums = [s1,s2,s3,s35,s4,s5,s6]
sides=[1]*s1+[2]*s2+[3]*s3+[3.5]*s35+[4]*s4+[5]*s5+[6]*s6
S = len(sides)
L=len(Lens)
Ns=sides[S-1]/Lens[0]
Ns=math.ceil(Ns)


#_____________________________________________________
#Adding Variables

#add variables for design blueprint
p = z.addVars(L,Ns,S, vtype=GRB.BINARY)
```

```
#add variables for packing
b = z.addVars(L,N,I,J, vtype=GRB.BINARY)
bInUse = z.addVars(J, vtype=GRB.BINARY)
bRowInUse = z.addVars(I,J, vtype=GRB.BINARY)


#_____________________________________________________
#constraints

#_____________________________________________________
#Design Blueprint Constraints

#implicit constraint
#Design Blueprint Constraint 1
for l in range(L):
   for s in range(S):
       sum = 0
       for n in range(N):
           sum = sum + p[l,n,s]
       z.addConstr(sum <= 1)


#side lengths are all met
#Design Blueprint Constraint 2
for s in range(S):
   sum = 0
   for l in range(L):
       for n in range(Ns):
           sum += p[l,n,s]*Lens[l]*n
   z.addConstr(sum == sides[s])


#available pipes are not exceeded
#Design Blueprint Constraint 3
for l in range(L):
   sum = 0
   for s in range(S):
       for n in range(Ns):
           sum += p[l,n,s]*n
   z.addConstr(sum <= avail[l])

#_____________________________________________________
#Packing Constraints

#implicit constraint
```

```
#Packing Constraint 1
for l in range(L):
   for i in range(I):
       for j in range(J):
           sum = 0
           for n in range(N):
               sum = sum + b[l,n,i,j]
           z.addConstr(sum <= 1)


#ifBagNotInUse (if bInUse[j]=0), then b[l,n,i,j]=0
#Packing Constraint 2
for j in range(J):
   for i in range(I):
       for n in range(N):
           for l in range(L):
               sum += b[l,n,i,j]
               z.addConstr((bInUse[j] == 0) >> (b[l,n,i,j] == 0))
               z.addConstr((b[l,n,i,j] == 1) >> (bInUse[j] == 1))


#ifRowNotInUse (if bRowInUse[j]=0), then b[l,n,i,j]=0
#Packing Constraint 2
for j in range(J):
   for i in range(I):
       for n in range(N):
           for l in range(L):
               z.addConstr((bRowInUse[i,j] == 0) >> (b[l,n,i,j] == 0))
               z.addConstr((b[l,n,i,j] == 1) >> (bRowInUse[i,j] == 1))


#spatial constraint
#Packing Constraint 3
for i in range(I):
   for j in range(J):
       length = 0
       for n in range(N):
           for l in range(L):
               length += b[l,n,i,j]*n*Lens[l]
       z.addConstr(length <= Lmax)


#weight constraint
#Packing Constraint 4
for j in range(J):
   weight = 0
```

```
   for i in range(I):
       for n in range(N):
           for l in range(L):
               weight = weight + b[l,n,i,j]*n*Lens[l]*rho
   z.addConstr(weight <= Wmax-Wempty)

#_____________________________________________________
#Combining Design Blueprint and Packing Constraint

#Counting number of pipes of each length used in Design Blueprint
nums = [0]*L
for l in range(L):
   sum = 0
   for s in range(S):
       for n in range(Ns):
           sum += n*p[l,n,s]
   nums[l] = sum

#Number of pipes in design blueprint is same as number in packing
#Combine Constraint 1
for l in range(L):
   sum = 0
   for i in range(I):
       for n in range(N):
           for j in range(J):
               sum += b[l,n,i,j]*n
   z.addConstr(sum == nums[l])


#defining objective function
#_____________________________________________________


#counting number of bags used
nB = 0.0
for j in range(J):
   nB += bInUse[j]


#counting number of rows used
nRow = 0.0
for j in range(J):
   for i in range(I):
       nRow += bRowInUse[i,j]
```

```
#counting number of pipes used
nPipes = 0
for s in range(S):
    for l in range(L):
        for n in range(Ns):
            nPipes += p[l,n,s]*n


z.setObjective(1000*nB+10*nRow+nPipes, GRB.MINIMIZE)
z.optimize()

#_____________________________________________________
#Outputting Solution

print()
print(f"SOLUTION:")
print()

#Outputting Design Blueprint
print()
print(f"Set Design Blueprint:")
print()
sideIndex = 0
for sideLength in range(L):
    if sideNums[sideLength] > 0:
        print(f"{sideNums[sideLength]} sides of length {Lens[sideLength]}' are needed in the
        print(f"These sides are made up of the following combinations of pipes:")
        for index in range(sideNums[sideLength]):
            print(f"    ____________________________")
            for l in range(L):
                for n in range(Ns):
                    if p[l,n,index+sideIndex].X == 1:
                        print(f"    {n} pipe(s) of length {Lens[l]}'")
        print(f"    ----------------------------")
        sideIndex += sideNums[sideLength]




#Outputting packing configuration
print()
print(f"Bag Packing List and Configuration:")
print()
print(f"____________________________")
for l in range(L):
   sum = 0
```

```
   for s in range(S):
       for n in range(Ns):
           sum += n*p[l,n,s].X
   print(f"{sum} pipes of length {Lens[l]}'")
print(f"_____________________________")


#Finding total pipe length in and Weight of Bags
totalLengthInBag = [0]*J
totalWeightOfBag = [Wempty]*J
for j in range(J):
    for i in range(I):
        for l in range(L):
            for n in range(N):
                totalLengthInBag[j] += b[l,n,i,j].X * n * Lens[l]
                totalWeightOfBag[j] += b[l,n,i,j].X * n * Lens[l] * rho

#Printing the packing output
bagNum = 1
counts = [0]*len(Lens)
for j in range(J):
   if bInUse[j].X == 1:
       print(f"_____________________________")
       print(f"Bag {bagNum}:")
       print()
       print(f"Total Weight: {round(totalWeightOfBag[j],1)} pounds")
       print(f"Percent of Volume Used: {round(100*totalLengthInBag[j]/(I*Lmax))}%")
       print()
       bagNum += 1
       rowNum = 1
       for i in range(I):
           if bRowInUse[i,j].X == 1:
                print(f"    Row {rowNum}")
                rowNum += 1
                for l in range(L):
                    for n in range(N):
                        if b[l,n,i,j].X == 1:
                            print(f"        {n} pipe(s) of length {Lens[l]}'")
                            counts[l] += n
print(f"_____________________________")
#testing to make sure solution has right number of pipes included
#print(counts)
```